



**Tertiary Infotech Academy Pte Ltd**

UEN: 201200696W

# **LEARNER GUIDE**

For

## **Voice and Video Agents with n8n (C500)**

Course Code: C500

Conducted by

**Tertiary Infotech Academy Pte Ltd**

UEN: 201200696W

**Version 1.0**

## DOCUMENT VERSION CONTROL RECORD

| Version Number | Effective Date of Release | Summary of Included Changes                                                                     | Author                                  |
|----------------|---------------------------|-------------------------------------------------------------------------------------------------|-----------------------------------------|
| 1.0            | 11 August 2026            | Initial aligned release: concept-first PPT, Learner Guide, Lesson Plan and four connected labs. | Tertiary Infotech Academy Training Team |

## TABLE OF CONTENTS

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>Introduction.....</b>                                   | <b>4</b>  |
| <b>Course Learning Outcomes.....</b>                       | <b>4</b>  |
| <b>Before You Start — Preparation.....</b>                 | <b>4</b>  |
| <b>Topic 01 — Voice Agent.....</b>                         | <b>6</b>  |
| Lab 1 — Build a Listen-Think-Talk Web Voice Concierge..... | 12        |
| Lab 2 — Connect the Concierge to Telegram Voice Notes..... | 17        |
| <b>Topic 02 — Video Agent.....</b>                         | <b>23</b> |
| Lab 3 — Generate a Talking-Head Welcome Video.....         | 29        |
| Lab 4 — Automate a Governed Video Campaign Pipeline.....   | 34        |
| <b>Wrap-Up and Authoritative References.....</b>           | <b>41</b> |
| <b>Next Steps.....</b>                                     | <b>42</b> |
| <b>Glossary.....</b>                                       | <b>42</b> |

## Introduction

This guide teaches the architecture and operating decisions behind voice and video agents before walking through the n8n builds. The connected HarbourStay scenario begins with a browser voice concierge, reuses the same conversation core through a Telegram voice-note adapter, then turns approved service facts into an avatar welcome video and a governed distribution pipeline.

Every external service is called with a learner-owned account and placeholder credentials. Provider interfaces and pricing can change, so confirm the current official documentation linked in the references, keep clips short during practice and retain the request, status and output evidence described in each lab.

## Course Learning Outcomes

- LO1: Explain the listen-think-talk architecture of a voice agent, select suitable business use cases, and design the n8n workflow and channel boundary.
- LO2: Build and verify an n8n voice agent that transcribes audio with Whisper, reasons with an AI Agent, speaks with ElevenLabs, and operates through browser and chat channels.
- LO3: Explain avatar-video architecture and build an n8n workflow that turns a governed brief into a talking-head video through an asynchronous video API.
- LO4: Automate a monitored video-production and distribution pipeline with approval, retry, accessibility, privacy, cost and human-handoff controls.

## Before You Start — Preparation

### What you need

- A laptop with a current Chromium-based browser and permission to use the microphone.
- An n8n Cloud workspace or a recent self-hosted n8n instance reachable from the browser and Telegram.
- An OpenAI API project with enough credit for short Whisper transcription and chat-model practice.
- An ElevenLabs account with an authorised voice and enough quota for short text-to-speech clips.
- A HeyGen account with API access, an authorised stock or custom avatar and voice, and quota for short test renders.
- A Telegram bot created through BotFather and, for Lab 4 distribution, a private test channel where the bot is an administrator.
- Python 3 is available to serve the browser lab apps; on Windows, the py launcher may be used when python is not on PATH.

- The C500 repository cloned locally; do not commit provider keys, tokens, generated customer data or private media.

### Verify your setup

Open n8n, create a blank workflow and confirm that Webhook, HTTP Request, AI Agent, OpenAI Chat Model, Crypto, Data Table, Telegram, Wait and IF nodes are available. Import each starter JSON before editing it.

```
python --version
```

```
# Windows fallback: py --version
```

```
git clone https://github.com/tertiarycourses/C500-Voice-and-Video-Agents-with-n8n.git
```

```
cd C500-Voice-and-Video-Agents-with-n8n
```

```
git status --short
```

### Conventions used in every lab

- Replace placeholders such as <OPENAI\_HEADER\_AUTH>, <ELEVENLABS\_HEADER\_AUTH>, <HEYGEN\_HEADER\_AUTH>, <TELEGRAM\_CREDENTIAL> and <CHANNEL\_ID> inside n8n with your own credential selections or test values.
- Use n8n credential storage for secrets. Do not paste API keys into prompts, workflow names, Sticky Notes, lab evidence or Git-tracked files.
- Use the Webhook Test URL while the workflow is listening in the editor; activate the workflow before changing an app to its Production URL.
- Keep every practice recording and video synthetic, short and free of real guest, employee or account information.
- A successful HTTP status is intermediate evidence; complete the lab only after the stated media output and failure-path check both succeed.

## Topic 01 — Voice Agent

AI voice agents and n8n | webhooks and AI Agent nodes | Whisper and ElevenLabs | browser, chat and phone patterns | service use cases

### Key concepts

- A voice agent is a conversational system that accepts speech, derives text and intent, chooses a bounded response, then renders speech; audio alone does not make a workflow agentic.
- n8n orchestrates triggers, binary audio, model calls, credentials, branches, retries and channel delivery while external models perform transcription, reasoning and speech synthesis.
- Speech-to-text converts an audio signal into a transcript; the AI Agent interprets that transcript against role, context and policy rather than treating it as an instruction with unlimited authority.
- Text-to-speech converts approved response text into audio; voice identity, pronunciation, language, pace and output format are production parameters.
- Browser, messaging and telephone channels expose different audio formats, session identifiers, time limits and response contracts, so the conversation core should be separated from channel adapters.
- Perceived quality depends on the full latency budget: capture, upload, transcription, reasoning, synthesis and delivery all contribute to the pause a caller experiences.
- Conversation state needs an explicit key, bounded memory and expiry; channel identifiers are useful routing data but should not silently become long-term personal profiles.
- A safe voice agent discloses its automated nature, minimises recordings and transcripts, validates callers before sensitive actions, logs decisions, and hands off when confidence or authority is insufficient.

### Voice Agent or Voice Automation?

A recorder, transcription job or fixed menu may use speech without being an agent. A voice agent interprets a conversational goal, chooses among permitted actions and continues until it can answer, clarify or hand off. Use the simplest pattern that satisfies the service need.

HarbourStay begins with frequently asked guest questions because the answers are bounded and a wrong reply is reversible. Payments, identity changes, emergency advice and contractual commitments stay with authorised staff even if the same voice interface receives the request.

- Transcribe: Speech becomes text; no decision is made.
- Route: A known intent selects a fixed workflow branch.
- Agent: A bounded model chooses a response or tool.

- Human: Consequential or uncertain requests leave automation.

### The Listen-Think-Talk Loop

Listen captures audio, validates the file and transcribes it. Think applies the role, approved knowledge, conversation context and available tools. Talk converts only the approved response into audio and returns it through the originating channel.

Verification closes the loop. The workflow checks that every stage returned usable evidence, that no unapproved action occurred, and that the final audio corresponds to the response text. A failure at one stage must not be disguised by a fluent later stage.

- Listen: Capture -> validate -> transcribe.
- Think: Interpret -> retrieve -> decide.
- Talk: Approve text -> synthesise -> deliver.
- Verify: Trace stages, evidence and fallback.

### n8n as the Conversation Orchestrator

The Webhook or Telegram Trigger starts a run. Core nodes normalise channel-specific fields, provider nodes or HTTP Request nodes call the models, the AI Agent applies the role, and a response node honours the channel contract. Credentials remain in n8n credential storage.

Keep provider calls in named subflows or clearly labelled nodes. This makes models replaceable, errors observable and usage measurable. The workflow is responsible for routing and control even when the language model supplies the wording.

- Trigger: Receive audio and a stable conversation key.
- Normalise: Produce one transcript request schema.
- Conversation Core: Apply role, context and response policy.
- Adapter: Return browser audio, chat audio or call markup.

### Audio Transport and Binary Data

Audio reaches n8n as binary data, a downloadable file identifier or a provider URL. The workflow must preserve the binary field name, MIME type and filename expected by the transcription call. A JSON body that merely names a local file cannot carry that file across the network.

Browser MediaRecorder commonly produces WebM audio, Telegram voice notes use OGG/Opus, and telephone platforms may use streamed or provider-hosted audio. Validate size and type early, then convert only when the chosen provider requires it.

- Browser: Multipart upload from MediaRecorder.
- Telegram: File ID -> download -> binary property.
- Phone: Call event plus media stream or recording URL.

- Provider: Accepted format, limit and response schema.

### Whisper Transcription Is Evidence, Not Intent

The transcription model estimates words from audio. Noise, accents, product names and code-switching can change the transcript. Preserve the observed transcript and language metadata so later failures can be traced to listening rather than reasoning.

Do not let a transcript directly trigger a high-impact action. Normalise the text, detect empty or implausible results and ask the speaker to repeat when needed. Domain vocabulary can be supplied as context, but the agent still needs confirmation for names, dates and quantities.

- Audio Check: Type, size, duration and signal are usable.
- Transcribe: Whisper returns text and optional metadata.
- Normalise: Trim, label language and keep provenance.
- Clarify: Repeat critical facts instead of guessing.

### AI Agent Role and Response Contract

A production system message names the organisation, authorised topics, style, sources, forbidden actions and escalation route. It also defines a machine-readable response contract so downstream nodes can distinguish an answer from a handoff or clarification.

Source content and caller speech are untrusted data. They may describe instructions but cannot expand tools, reveal hidden prompts or waive policy. The AI Agent should state limitations and return a safe handoff when the required evidence is not available.

- Role: Who the agent represents and serves.
- Scope: Topics and actions it may handle.
- Response: Answer, clarify or handoff with reason.
- Evidence: Source, timestamp and uncertainty retained.

### ElevenLabs Speech Synthesis

The text-to-speech request combines approved text, a voice identifier, a model and output settings. The response is binary audio; n8n must preserve it through to the browser or messaging node. Synthesising before policy checks risks giving unsafe text a persuasive voice.

Choose a voice that is intelligible for the language and context. Test names, abbreviations, numbers and URLs. Short sentences and spoken punctuation improve comprehension, while excessive expressiveness can be inappropriate for service or safety messages.

- Input: Approved response text only.
- Voice: Authorised voice ID and language fit.

- Model: Quality, latency and cost choice.
- Output: Audio format suited to the channel.

### Channel Adapters: Browser, Chat and Phone

A browser can wait for one HTTP response and play returned audio. A chat platform delivers separate events and files, so the workflow replies to a chat identifier. A telephone platform adds call lifecycle events, strict response timing and telephony-specific markup or streaming.

Keep the conversation core independent of these differences. Each adapter maps its inbound event to transcript, session and caller context, then maps answer, clarification or handoff back to the channel. This avoids duplicating policy across three workflows.

- Browser: Webhook in; audio response out.
- Chat: Message event; file download; reply to chat.
- Phone: Call event; low-latency turn or callback.
- Core: Same role, policy and knowledge contract.

### Latency Budget and Conversation Turn Design

Silence feels longer in speech than in a web form. Measure capture upload, transcription, reasoning, synthesis and delivery separately. Faster models, shorter prompts and shorter answers help only when that stage is the real bottleneck.

For slow paths, acknowledge the request or move to an asynchronous channel rather than leaving the caller in silence. Streaming can reduce perceived delay, but it also makes moderation and interruption handling harder, so use it deliberately.

- Capture: End-of-speech detection and upload.
- Listen: Transcription provider time.
- Think: Context load, tools and model generation.
- Talk: Synthesis plus channel delivery.

### Conversation State, Memory and Idempotency

A session key binds related turns. Store only the context needed for continuity, apply an expiry and separate durable customer records from transient conversation memory. A channel username or chat ID is not proof of identity for protected information.

Events can be retried. Use provider event IDs or a derived idempotency key to prevent duplicate bookings, messages or usage charges. The agent should be able to repeat an informational answer without repeating a side effect.

- Session: Stable key for related turns.
- Memory: Minimum context with retention limit.
- Identity: Separate verification before protected actions.

- Idempotency: One side effect per event or request.

### Failures, Fallbacks and Human Handoff

Transcription can be empty, models can time out, speech synthesis can fail and channels can reject a file. Classify each error, retry only transient failures and preserve a plain-text fallback where the channel permits it.

A useful handoff includes the caller's stated goal, transcript, steps already attempted, relevant evidence and consented contact route. It must not expose credentials or hidden instructions. The human owner needs enough context to continue without forcing the customer to start again.

- Retry: Bounded backoff for transient provider errors.
- Clarify: Ask again when speech or intent is unclear.
- Fallback: Return text or a safe service message.
- Handoff: Transfer context to a named human queue.

### Privacy, Consent and Voice Safety

Tell people they are interacting with an automated system and whether audio or transcripts are retained. Collect only what the service needs, protect credentials in n8n and remove recordings according to a documented retention rule.

Never clone a person's voice without documented authority. Confirm identity before exposing booking or account data, block sensitive data from logs, and maintain an immediate route to a person for emergencies, complaints and vulnerable users.

- Disclose: Automated identity and recording notice.
- Minimise: Least audio, transcript and profile data.
- Authorise: Approved voice, data and tool permissions.
- Escalate: Human route for consequential requests.

### Tool Authority and Transaction Boundaries

Conversation and action are different privileges. An agent may explain a policy without being allowed to change a booking, charge a card or reveal an account record. Give each tool one narrow purpose, validated inputs and a named owner.

Before an irreversible or sensitive action, verify identity outside the model, restate the proposed change and require explicit confirmation. Record the tool input and result so fluent speech never becomes the only evidence of a transaction.

- Inform: Answer from approved service facts.
- Prepare: Draft an action without committing it.
- Confirm: Verify identity and explicit intent.
- Execute: Use one authorised tool and log its result.

## Voice Prompt Injection and Untrusted Speech

A caller can speak instructions such as 'ignore your rules' or ask for hidden configuration. Treat the transcript as user data, not as a system message. The role, policy and tool permissions remain outside the caller-controlled content.

Retrieved pages and transcripts can contain the same attack. Separate quoted evidence from instructions, allow-list tools and route attempts to reveal secrets or expand authority to a safe refusal or human review.

- Separate: System policy is not caller-controlled text.
- Allow-list: Only named tools and actions are reachable.
- Validate: Check arguments before every side effect.
- Escalate: Refuse or hand off suspicious requests.

## Language, Pronunciation and Inclusive Speech

Language detection, accent and code-switching can change both transcription and synthesis. Supply domain terms where supported, confirm critical names or numbers, and test the same service facts with representative speakers rather than assuming one clean recording.

Write for listening: short sentences, expanded abbreviations and natural pauses. Provide a text alternative when speech is hard to hear, and never use vocal style to imply a human identity or certainty the system does not have.

- Detect: Observe language; do not guess silently.
- Confirm: Repeat names, dates and quantities.
- Pronounce: Test domain terms and abbreviations.
- Include: Offer text and human alternatives.

## Voice-Agent Observability

A successful HTTP response does not prove a useful conversation. Track transcription success, route choice, handoff rate, stage latency, provider errors and the share of answers grounded in approved facts. Keep request IDs consistent across stages.

Review sampled failures with redacted evidence. Rising clarification rates may indicate noisy audio or missing vocabulary; rising handoffs may indicate a knowledge gap; slow synthesis may call for a different voice model rather than a larger language model.

- Quality: Grounded answer and clarification rates.
- Safety: Handoffs, refusals and blocked actions.
- Latency: Listen, think, talk and delivery time.
- Cost: Usage per completed useful turn.

## Phone Deployment Decision Record

A telephone pilot needs more than replacing the browser webhook. Record the provider event schema, media transport, call identifier, response deadline, disclosure, verified transfer route and behaviour when the caller interrupts or disconnects.

Name the regions, hours and request types permitted for the pilot. Keep payments, emergency advice and protected account changes outside scope until identity, recording consent, monitoring and rollback have been separately approved.

- Contract: Events, media and response deadline.
- Identity: Verified factor before protected data.
- Transfer: Named queue with failure behaviour.
- Rollback: Disable route and preserve evidence.

## Worked Trace: HarbourStay Late Check-In

A guest asks, 'Can I arrive after midnight?' The browser adapter uploads one WebM recording. Whisper returns the observed transcript. The concierge role answers from the supplied service facts and does not claim to change the booking. ElevenLabs renders the approved response.

The verification record contains the request ID, audio type, transcript, response text, route, provider status and latency per stage. If the guest asks to change a named reservation, the agent requests a verified handoff instead of treating the spoken name as sufficient identity.

- Request: Audio plus request ID; no credentials in payload.
- Transcript: Observed question retained for traceability.
- Decision: Information answer; no booking side effect.
- Delivery: MP3 returned; trace proves each stage.

## Lab 1 — Build a Listen-Think-Talk Web Voice Concierge

Learning outcome: LO2: Build and verify an n8n voice agent that transcribes audio with Whisper, reasons inside a bounded concierge role, and speaks through ElevenLabs.

Goal: You build the reusable conversation core for HarbourStay. A browser records a short WebM clip and sends it to an n8n Webhook. The workflow validates the binary input, calls OpenAI Whisper for transcription, gives the observed text to an AI Agent, converts the approved response through ElevenLabs and returns playable MP3 audio. The lab ends with both a normal information request and a request that must be handed to a person.

Suggested duration: 55 minutes.

## Prerequisites

- The C500 repository is available locally and labs/apps/voice-console/index.html opens in a current Chromium-based browser.
- An n8n workspace is reachable from the browser; if it is self-hosted, its public HTTPS URL is reachable by the browser used for testing.
- Learner-owned OpenAI and ElevenLabs credentials have quota for two short audio turns and are not stored in any Git-tracked file.
- An authorised ElevenLabs voice ID is available; the course does not require or permit cloning a person's voice.

## What you'll build

An active c500-voice-concierge webhook, a configured browser voice console, one verified spoken FAQ response, one verified human-handoff response, and an evidence record containing the request ID, transcript, answer, route and stage timings without secrets or real customer data. (Tools: n8n Webhook, HTTP Request, OpenAI Whisper, AI Agent, OpenAI Chat Model, ElevenLabs TTS, Respond to Webhook, browser MediaRecorder.)

## Step-by-step

1. Create an evidence folder outside the public repository, or use an ignored local evidence/ folder. Copy labs/evidence-template.md to evidence/lab01-voice-core.md. Record only synthetic prompts and redacted provider identifiers.

# Windows PowerShell

```
New-Item -ItemType Directory -Path evidence -Force
```

```
Copy-Item labs\evidence-template.md evidence\lab01-voice-core.md
```

# macOS / Linux / WSL2

```
mkdir -p evidence
```

```
cp labs/evidence-template.md evidence/lab01-voice-core.md
```

2. In n8n, choose Workflows -> Create Workflow -> menu -> Import from File and import the supplied starter. Rename it C500 Lab 01 - Voice Concierge if the imported name differs.

labs/workflows/lab01-voice-concierge.json

3. Open Voice Webhook. Confirm POST, path c500-voice-concierge, response using Respond to Webhook, and binary property data. Copy the Test URL but do not activate the workflow yet.

POST /webhook-test/c500-voice-concierge

Binary field: data

4. Create or select an n8n HTTP Header Auth credential for OpenAI. The credential header name is Authorization and its value is Bearer followed by your learner-owned key. Select it in Transcribe with Whisper; do not paste the key into the URL, body or Sticky Note.

Header name: Authorization

Header value: Bearer <OPENAI\_API\_KEY>

5. Inspect Transcribe with Whisper. Confirm POST <https://api.openai.com/v1/audio/transcriptions>, multipart-form-data, binary parameter file from input field data, and text parameter model set to whisper-1. Set the response format to JSON.

file = binary field data

model = whisper-1

6. Open Normalise Transcript. Confirm it reads the original Voice Webhook header X-C500-Request-ID when supplied, otherwise derives one from the execution ID; trims the returned text; and rejects an empty transcript before the AI Agent.

```
request_id = {{$('Voice Webhook').item.json.headers['x-c500-request-id'] || ('c500-' + $execution.id)}}
```

```
transcript = {{$json.text.trim()}}
```

7. Create or select the OpenAI credential used by the OpenAI Chat Model node and attach that model to Concierge AI Agent through the AI Language Model connector. Use the current low-latency chat model available to your account and set temperature low where the node exposes it.

Credential: <OPENAI\_CHAT\_CREDENTIAL>

Model: a current low-latency chat model available to your project

8. Read the Concierge AI Agent system message. Keep the HarbourStay facts, the maximum spoken-answer length, the answer/clarify/handoff routes and the rule that speech or source text cannot expand authority. Do not add booking, payment or personal-data tools.

Allowed facts:

- Reception is staffed 24 hours.
- Standard check-in begins at 3:00 pm.
- Breakfast is served 7:00-10:30 am.
- Booking changes require a verified staff handoff.

- 9.** Open Prepare Spoken Reply. Confirm it produces response\_text and route, strips Markdown formatting, limits the spoken text to 420 characters and retains the request\_id and transcript for the evidence record.

```
route in [answer, clarify, handoff]
```

```
response_text length <= 420
```

- 10.** Create or select an n8n HTTP Header Auth credential for ElevenLabs with header name xi-api-key. Select it in Synthesise with ElevenLabs. Keep the API key out of node parameters and exported JSON.

```
Header name: xi-api-key
```

```
Header value: <ELEVENLABS_API_KEY>
```

- 11.** Open Voice Configuration. Replace <ELEVENLABS\_VOICE\_ID> with an authorised stock or organisational voice ID. Keep model\_id eleven\_multilingual\_v2 unless the current ElevenLabs documentation and your account require another supported model.

```
voice_id = <ELEVENLABS_VOICE_ID>
```

```
model_id = eleven_multilingual_v2
```

- 12.** Inspect Synthesise with ElevenLabs. Confirm POST to the text-to-speech endpoint for the configured voice ID, JSON body with text and model\_id, Accept audio/mpeg, and Response Format File written to binary property data.

```
POST https://api.elevenlabs.io/v1/text-to-speech/{{$.json.voice_id}}
```

```
Response binary property: data
```

- 13.** Inspect Return Spoken Reply. Confirm Respond With Binary File, input binary field data, and response header Content-Type audio/mpeg. Save the workflow.

```
Content-Type: audio/mpeg
```

```
Content-Disposition: inline; filename=c500-reply.mp3
```

- 14.** Open labs/apps/voice-console/index.html through a local server, paste the n8n Test URL into Webhook URL and click Save URL. Keeping the n8n editor listening, record: What time can I check in if I arrive late tonight? Stop after one sentence and send.

```
# Windows, macOS or Linux
```

```
python -m http.server 8000 --directory labs/apps/voice-console
```

```
Open http://localhost:8000
```

- 15.** Confirm the page reports an HTTP 200 response and plays an MP3 answer. In the n8n execution, inspect each node and record the request ID, audio MIME type,

transcript, route, response text and approximate stage timestamps in evidence/lab01-voice-core.md. Do not copy credentials or complete provider response headers.

Expected route: answer

Expected facts: 24-hour reception and check-in from 3:00 pm

**16.** Run the bounded failure path by asking: Change the booking for Alex Tan to tomorrow. Confirm the agent does not claim to find or change a booking and returns a verified-staff handoff message. Record the route and wording.

Expected route: handoff

Forbidden result: a claimed booking change

**17.** Activate the workflow, copy its Production URL into the browser console and repeat the normal question. Confirm the production execution succeeds, then stop the local web server when finished.

POST /webhook/c500-voice-concierge

## Test it

The production browser console sends a WebM recording and plays an audio/mpeg reply. The execution shows non-empty Whisper transcript evidence, one bounded Concierge AI Agent result and ElevenLabs binary audio. The late-check-in question uses only approved facts, while the named-booking request routes to handoff and causes no external action. The evidence file contains request, route and timing data but no secret or real customer data.

## Checkpoint

Keep the active production webhook URL only in the local browser console, export the credential-free workflow if you changed its structure, and keep the authorised voice ID plus HarbourStay service facts ready for Lab 2. Rejoin here: import labs/workflows/lab01-voice-concierge.json, select the OpenAI Header Auth, OpenAI Chat Model and ElevenLabs Header Auth credentials, set Voice Configuration, then start at the normal-turn test.

## Troubleshooting

- Webhook shows no binary data — Confirm the console sends multipart/form-data with field name data and the Webhook receives the request while listening on the Test URL.
- Whisper returns unsupported format or an empty transcript — Record a new short clip in a Chromium-based browser, confirm MIME type audio/webm, and keep the microphone close enough for clear speech.

- ElevenLabs returns 401 or 404 — Reselect the Header Auth credential, verify the authorised voice ID in Voice Configuration and confirm the learner account has access to that voice.
- Browser receives audio but does not play it — Verify Respond to Webhook sends binary field data with Content-Type audio/mpeg and inspect the browser network response for a non-zero payload.

### Challenge

Add an optional language field to the browser console and pass it as transcription context and response-language guidance. Keep the same safety route and prove one non-English information question without adding a new authority.

### Reflection

Which stage produced the largest observed delay, and what single change would reduce it without weakening the role, evidence or handoff controls?

**Note:** Full commands are in labs/lab-01-\*.md. Use only synthetic HarbourStay data, voices, avatars, bots and channels that you are authorised to use. Keep secrets in n8n credentials, disclose synthetic media, and do not publish a generated asset outside the private training channel.

## Lab 2 — Connect the Concierge to Telegram Voice Notes

Learning outcome: LO2: Reuse the verified conversation core through a chat-channel adapter and map the additional controls required for a telephone channel.

Goal: You separate channel-specific handling from the conversation core built in Lab 1. Telegram supplies a chat event and voice file ID rather than a browser upload, so the workflow validates the event, downloads the OGG/Opus file, transcribes it, applies the same concierge role, synthesises audio and sends the file back to the originating chat. You also write a phone-channel map that makes identity, call state, response timing and human transfer explicit instead of treating a phone number as a drop-in replacement.

Suggested duration: 45 minutes.

### Prerequisites

- Lab 1 is complete and the bounded HarbourStay concierge system message, service facts and authorised ElevenLabs voice ID are available.
- A learner-owned Telegram bot exists, its token is stored only in an n8n Telegram credential, and the bot is used in a private training chat.
- OpenAI and ElevenLabs credentials from Lab 1 remain available in n8n and have quota for two short turns.

- The learner can activate an n8n workflow at a public HTTPS URL so Telegram can deliver updates.

## What you'll build

An active Telegram voice-note concierge with a text-message help branch, bounded four-turn memory keyed to the chat, one verified spoken reply, one verified fallback, and phone-channel-map.md covering call event, media, session, response and transfer contracts. (Tools: n8n Telegram Trigger and Telegram nodes, OpenAI Whisper, AI Agent, Window Buffer Memory, ElevenLabs TTS, IF, chat and phone adapter design.)

## Step-by-step

1. Copy the evidence template to evidence/lab02-telegram-channel.md and create evidence/phone-channel-map.md. Keep Telegram tokens, usernames and personal chat identifiers out of both files.

# Windows PowerShell

```
Copy-Item labs\evidence-template.md evidence\lab02-telegram-channel.md
```

```
New-Item -ItemType File -Path evidence\phone-channel-map.md -Force
```

# macOS / Linux / WSL2

```
cp labs/evidence-template.md evidence/lab02-telegram-channel.md
```

```
: > evidence/phone-channel-map.md
```

2. Import the supplied workflow and rename it C500 Lab 02 - Telegram Voice Agent. Open Telegram Trigger and select your learner-owned Telegram credential. Leave updates set to Message.

```
labs/workflows/lab02-telegram-voice-agent.json
```

3. Inspect Has Voice Note. Confirm the true condition checks that message.voice.file\_id exists. The false branch must send one help message and stop; it must not call Whisper, the AI Agent or ElevenLabs for an ordinary text message.

```
True when: {{$json.message.voice.file_id}} is not empty
```

```
False reply: Send a short voice note to use this training bot.
```

4. Open Download Voice Note. Select the same Telegram credential, resource File, operation Get, File ID from Telegram Trigger message.voice.file\_id, Download enabled and binary property data.

```
File ID: {{$('Telegram Trigger').item.json.message.voice.file_id}}
```

```
Download: true
```

Binary: data

5. Open Transcribe with Whisper. Reuse the OpenAI Header Auth credential from Lab 1. Confirm multipart file uses binary property data and model is whisper-1. Preserve the Telegram chat ID only for routing and session; do not send it to the transcription provider as prompt text.

file = binary field data

model = whisper-1

6. Inspect Prepare Chat Turn. Confirm it produces transcript, response\_chat\_id and request\_id from the Telegram event. Voice Configuration then derives a session key from the private chat ID plus a resettable training epoch; do not treat the chat ID as verified customer identity.

response\_chat\_id = Telegram message.chat.id

request\_id = tg-update-<update\_id>

7. Open Voice Configuration. Replace <ELEVENLABS\_VOICE\_ID> with the same authorised voice used in Lab 1, keep session\_epoch practice-a, and select the ElevenLabs Header Auth credential in Synthesise with ElevenLabs.

voice\_id = <ELEVENLABS\_VOICE\_ID>

model\_id = eleven\_multilingual\_v2

session\_epoch = practice-a

8. Attach the OpenAI Chat Model and Window Buffer Memory to Concierge AI Agent. Set memory Session Key to the exact Voice Configuration expression and context window length to 4. Keep the same role, facts and routes as Lab 1.

Session key: tg-<response\_chat\_id>-<session\_epoch>

Context window: 4

9. Inspect Send Spoken Reply. Select the Telegram credential, resource Message, operation Send Audio, Chat ID from response\_chat\_id, Binary File enabled and Input Binary Field data. Disable automatic attribution if the node exposes that option.

Chat ID: {{\$('Prepare Chat Turn').item.json.response\_chat\_id}}

Operation: Send Audio

Binary field: data

10. Save and activate the workflow. In the private chat with your bot, send a three-to-five-second voice note: When is breakfast served? Wait for a spoken reply.

Expected fact: breakfast is served 7:00-10:30 am

11. Inspect the production execution. Confirm the Telegram Trigger, file download, Whisper transcript, agent route, ElevenLabs binary and Send Audio nodes all

succeeded. Record only the redacted update suffix, transcript, route and elapsed time in evidence/lab02-telegram-channel.md.

Expected route: answer

Expected output: one audio message in the originating chat

**12.** Send the text message hello. Confirm only the false branch runs and the bot asks for a voice note. Verify that no transcription, language-model or synthesis provider call occurred for this event.

Expected: one text help reply; zero media-provider calls

**13.** Send two connected voice turns: What time is breakfast? followed by Which service did I ask about in my previous message? Confirm the four-turn memory identifies breakfast. Then change Voice Configuration session\_epoch from practice-a to practice-b and repeat only the second question; the clean session has no previous message and must clarify instead of claiming remembered context.

Connected session: practice-a -> expected service is breakfast

Clean session: practice-b -> expected no prior-message context and clarification

**14.** Write evidence/phone-channel-map.md with a five-row table: Inbound event, Audio transport, Session key, Response contract and Human transfer. Map Telegram's current values and the values a chosen phone provider would require. Use placeholders rather than a real number or account identifier.

```
| Contract | Telegram training adapter | Future phone adapter |
|---|---|---|
| Inbound event | message update | call lifecycle webhook |
| Audio transport | voice file ID -> OGG download | <media stream or provider URL>
|
| Session key | private chat ID | <provider call ID> |
| Response contract | sendAudio to chat | <Twiml, stream or call API> |
| Human transfer | text/audio handoff notice | <verified queue and transfer action> |
```

**15.** Add four phone-only controls beneath the table: automated-agent disclosure, caller verification before protected data, maximum response deadline and an immediate named human-transfer path. State that a phone number alone is not verified identity.

- Disclosure: <opening notice>
- Verification: <approved factor before protected data>
- Deadline: <provider-specific response limit>

- Transfer: <named human queue and failure route>

## Test it

A private Telegram voice note produces exactly one spoken reply in the originating chat, two connected turns use the bounded session memory, and changing session\_epoch starts a deterministic clean session. A text-only update takes the help branch and causes zero Whisper, model or ElevenLabs calls. The redacted evidence identifies each stage, and phone-channel-map.md names the event, media, session, response and verified human-transfer contract without claiming that the Telegram workflow is already a telephone deployment.

## Checkpoint

Keep the Telegram bot and workflow private, preserve the shared concierge role and authorised voice settings, and retain the channel map as the design input for any separately approved telephone pilot. Rejoin here: import labs/workflows/lab02-telegram-voice-agent.json, select Telegram, OpenAI and ElevenLabs credentials, set voice\_id and session\_epoch, activate, then start at the first voice-note test.

## Troubleshooting

- Telegram Trigger does not receive updates — Activate the workflow, confirm the n8n public URL uses HTTPS, reselect the bot credential and ensure no second active workflow is consuming the same bot updates.
- Download Voice Note has no binary field — Check that the true branch received message.voice.file\_id, operation Get has Download enabled and the binary property is data.
- Send Audio reports missing binary data — Confirm ElevenLabs Response Format is File, binary property data, and no Edit Fields node discarded binary data before the Telegram node.
- The second turn ignores the first — Confirm Window Buffer Memory is connected to the AI Agent and the same derived session\_key reaches both runs; do not hard-code one shared key for all chats.

## Challenge

Add a /forget text command that clears only the current chat's bounded conversation memory and returns a confirmation without calling any media provider. Document how the user can invoke it.

## Reflection

Which parts of the conversation core remained unchanged across browser and Telegram, and which phone-specific responsibilities cannot safely be hidden inside that core?

**Note:** Full commands are in labs/lab-02-\*.md. Use only synthetic HarbourStay data, voices, avatars, bots and channels that you are authorised to use. Keep secrets in n8n credentials, disclose synthetic media, and do not publish a generated asset outside the private training channel.

### Topic recap

- You can now: LO1: Explain the listen-think-talk architecture, select bounded voice use cases, and define channel, tool and human-handoff boundaries.
- You can now: LO2: Build and verify a Whisper-to-AI-Agent-to-ElevenLabs workflow for browser and Telegram voice channels.

## Topic 02 — Video Agent

AI video agents and digital avatars | talking-head video APIs | asynchronous n8n production | marketing, training and service | governed distribution

### Key concepts

- A video agent turns an objective and approved source material into a planned, generated, checked and distributed media artifact; generation is one stage of the agent workflow.
- A talking-head request combines script, avatar or talking-photo identity, voice, language, dimensions and optional background or captions into a provider-specific job schema.
- Video generation is asynchronous: a create request returns a job identifier, while later status checks return processing, completed or failed state and eventually a media URL.
- The script is the highest-leverage control: it names audience, objective, claim sources, spoken length, call to action and forbidden claims before an avatar renders it persuasively.
- Consent and provenance must cover the avatar image, voice, music, logos and factual source material; synthetic media should be disclosed when context could mislead viewers.
- n8n makes the pipeline observable through request IDs, polling limits, error branches, usage logs, approvals and distribution adapters rather than an opaque sequence of API calls.
- Distribution is a separate decision from generation. A completed file can enter an approval queue, asset library, Telegram channel or social publishing adapter without granting the generator unlimited publishing rights.
- Video quality includes factual accuracy, pronunciation, captions, visual safety, aspect ratio, accessibility, rendering latency and cost per accepted asset, not just visual realism.

### Video Generator or Video Agent?

A generator turns a supplied script into media. A video agent starts from a business objective, prepares a bounded script, selects approved assets, submits a render, observes job state, applies release checks and routes the result. The surrounding decisions make the workflow agentic.

Use a fixed template when the message barely changes. Use an agent when the brief varies and the script needs bounded reasoning. Keep a person in the release path when the content contains claims, named individuals, regulated advice or broad public reach.

- Template: Known text and fixed media slots.
- Generator: Script and assets become a video.

- Agent: Plans, renders, observes and routes.
- Publisher: Separate authority releases the asset.

### Talking-Head Video Anatomy

The provider needs a visual identity, a voice or audio source, spoken text, scene settings and output dimensions. Some APIs use stock avatar IDs; others animate a talking photo. IDs are configuration values and should be validated before a production run.

The same script can produce very different results with a new avatar, voice, pace or aspect ratio. Store these choices as an approved campaign profile rather than scattering them across workflow expressions.

- Script: Approved spoken words and pronunciation cues.
- Avatar: Authorised visual identity or talking photo.
- Voice: Authorised voice ID, language and pace.
- Canvas: Aspect ratio, background and captions.

### HeyGen Asynchronous Job Lifecycle

The create-video request is accepted before rendering finishes and returns a video identifier. The workflow stores that identifier, waits, asks the status endpoint and branches on completed, failed or still-processing state. A successful HTTP response is not proof that a video exists.

Set a maximum number of checks and an overall deadline. Preserve provider error details, and avoid creating a second render merely because a status call was slow. The original request ID and video ID are the evidence needed for safe recovery.

- Create: Validate brief and submit one render job.
- Store: Keep request ID, video ID and submitted settings.
- Poll: Wait, check state and count attempts.
- Resolve: Completed URL, failure record or timeout queue.

### Script Contract: Brief to Spoken Copy

The script prompt names audience, outcome, source facts, spoken duration, tone, call to action and exclusions. Require a structured result with title, spoken script, caption and claim list so later checks do not need to infer what the model intended.

Spoken copy needs short sentences, explicit pronunciation and no invisible formatting. Estimate duration from word count, then enforce a maximum. A beautiful render should be rejected if the script exceeds the time box or introduces an unsupported claim.

- Audience: Who watches and what they already know.
- Outcome: One useful action after watching.
- Sources: Approved facts and offer wording.

- Constraints: Length, tone, claims and disclosure.

### Avatar and Voice Consent

Stock provider assets come with platform terms; custom likenesses and voices require documented authority from the represented person and the organisation. Consent should cover intended channels, territories, duration and withdrawal handling.

Do not use an avatar to imply that a real person personally delivered or endorsed content when they did not. Maintain asset provenance and a reviewable campaign profile so a future workflow run cannot silently switch identities.

- Owner: Who controls the image, voice and brand assets.
- Purpose: Permitted messages and audience.
- Term: Duration, channels and withdrawal process.
- Disclosure: When viewers must know media is synthetic.

### Campaign Profile and Asset Configuration

A campaign profile centralises avatar ID, voice ID, language, aspect ratio, logo, background, disclosure, destination and owner. The workflow reads this approved configuration instead of asking a language model to invent provider IDs.

Version the profile and validate it before submission. If a provider asset is removed, fail with a configuration message and route to the owner. Do not fall back to an arbitrary face or voice merely to complete the run.

- Identity: Avatar and voice IDs with provenance.
- Format: Dimensions, background, captions and language.
- Release: Owner, destination and disclosure rule.
- Version: Effective date and change reason.

### Polling, Timeouts and Idempotent Recovery

A Wait node spaces provider checks; an IF or Switch node interprets state; a loop counter prevents an endless run. Record the next permitted check time if the API communicates a rate limit, and route an exhausted deadline to an operator.

A retry of status is safe; a retry of creation may incur a second charge and duplicate asset. Create once per approved request key. Recovery resumes observation of the known video ID unless evidence proves the provider never accepted the original request.

- Wait: Respect render time and provider rate limits.
- Check: Completed, failed or processing state.
- Limit: Maximum attempts and elapsed deadline.
- Resume: Continue known job; do not duplicate creation.

## End-to-End Production Pipeline

The production pipeline separates intake, script preparation, approval, render, quality checks, storage and delivery. Each stage has an owner and an observable artifact. This makes a failed caption check recoverable without regenerating the script and avatar unnecessarily.

Use sub-workflows when the same rendering or distribution logic serves multiple campaigns. Pass a compact contract rather than the entire trigger payload, and return the video ID, final URL, checks and failure details.

- Prepare: Brief -> facts -> structured script.
- Approve: Claims, identity and destination checked.
- Render: Create once and observe job state.
- Release: Store, caption, distribute and log.

## Distribution Adapters and Release Authority

A completed provider URL is a candidate asset, not automatic permission to publish. The distribution layer verifies approval state, visibility, caption, destination identity and platform limits before upload or posting.

The C500 pipeline stores the asset record and demonstrates a Telegram channel adapter. The same contract can feed a content management or social API, but each adapter needs its own credentials, rate limits, moderation rules and rollback procedure.

- Asset Library: Stable record, provenance and retention.
- Approval Queue: Named reviewer and release decision.
- Channel: Caption, media limits and target identity.
- Rollback: Unpublish, correct and notify owner.

## Observability, Cost and Capacity

Track accepted requests, script approvals, create calls, completed renders, failed renders, timeouts, published assets and human interventions. Store timestamps for each stage so bottlenecks are visible rather than blamed on the final provider.

Cost per accepted video is more useful than cost per API call because rejected or duplicate renders consume budget without delivering value. Limit duration, resolution, retries and campaign frequency, and alert before a monthly cap is exhausted.

- Reliability: Completed renders / accepted requests.
- Latency: Median and tail time by stage.
- Quality: Accepted assets / completed renders.
- Cost: Provider spend / accepted assets.

## Accessibility and Release Quality

Captions support silent viewing and people who cannot rely on audio. Review reading speed, colour contrast, text placement and mobile aspect ratio. Supply a transcript or equivalent text alongside important training and service content.

Quality review compares the rendered words with the approved script, checks pronunciation and verifies that the image, voice, background and branding match the campaign profile. Accessibility and factual checks are release criteria, not optional polish.

- Words: Rendered speech matches approved script.
- Captions: Accurate, readable and time-aligned.
- Visual: Safe framing, contrast and correct identity.
- Alternative: Transcript or text equivalent available.

## Video Safety and Content Provenance

Save the source brief, facts, script version, asset profile, provider job ID, reviewer and distribution target. This provenance supports corrections and prevents a final URL from becoming an untraceable media object.

Block requests that impersonate people, fabricate testimonials, hide material disclosures or use unlicensed assets. External text can inform a script only after source authority is checked; it cannot command the workflow to skip review or change the destination.

- Source: Approved facts and content owner.
- Transform: Prompt, script and profile versions.
- Render: Provider, video ID and check results.
- Release: Reviewer, destination and published time.

## Moderation Before and After Rendering

Pre-render review checks the brief, claims, script, avatar, voice and destination before cost is incurred. Post-render review checks what viewers will actually receive: spoken words, captions, framing, visual artefacts and the final disclosure.

A pass at one checkpoint cannot replace the other. Provider rendering can introduce pronunciation or caption errors, while a polished final video can still be based on an unapproved claim or identity.

- Brief: Authority, source facts and audience.
- Script: Claims, length and disclosure approved.
- Render: Authorised identity and one job ID.
- Asset: Words, captions, framing and destination.

## Scene, Caption and Aspect-Ratio Design

A 16:9 training screen and a 9:16 mobile story have different safe areas. Keep faces, logos and captions within the target frame; limit on-screen text; and choose a background that preserves contrast without implying a location or endorsement that is not real.

Captions need readable line lengths and accurate timing. Treat dimensions, caption setting, background and safe-area rules as a versioned campaign profile so repeatable output does not depend on manual memory.

- Frame: Choose 16:9, 9:16 or 1:1 deliberately.
- Safe Area: Keep face, logo and text visible.
- Captions: Readable lines with accurate timing.
- Profile: Version dimensions and layout settings.

## Provider Portability and Stable Contracts

Video providers name avatars, voices, statuses and output fields differently. Keep one internal render request and result schema, then isolate provider-specific translation in named nodes or subflows.

A stable internal contract makes migration and testing easier. It also lets a failure branch describe one error class even when the provider changes its raw response, without hiding the original status needed for diagnosis.

- Request: Script, profile and idempotency key.
- Adapter: Translate to provider job schema.
- Status: Map processing, completed and failed.
- Evidence: Keep raw job ID and normalised result.

## Content Change Control

Approval must bind to the exact script, profile and destination reviewed by a person. Hash the script, version the campaign profile and store the approved destination; do not regenerate or silently substitute any of them after approval.

A later wording change creates a new snapshot and needs a new approval. The release workflow recomputes the hash and compares every value before rendering or publishing, making review evidence technically enforceable rather than ceremonial.

- Snapshot: Exact script and release metadata.
- Hash: Detect any wording change.
- Version: Bind avatar, voice and layout.
- Destination: Publish only to reviewed channel.

## Rollback and Media Incident Response

Rollback removes or disables the distributed asset while preserving the run record, source snapshot and provider job evidence. Deleting the audit trail makes it harder to understand the incident or prevent recurrence.

The runbook names an owner, reviewer, channel administrator and escalation contact. It defines how to stop the schedule, remove the private post, rotate compromised credentials and communicate a correction without starting another render blindly.

- Stop: Disable schedule and new releases.
- Contain: Remove or restrict the published asset.
- Preserve: Keep snapshot, job and decision evidence.
- Correct: Approve a new version before republishing.

## Worked Trace: HarbourStay Welcome Video

Marketing submits a welcome brief with three approved facts and a forty-five-second limit. The script node returns structured copy with no price claim. A reviewer approves the avatar, voice and destination. The create call returns one video ID and the workflow polls until completion.

Release checks compare the spoken message with the approved script, confirm captions and disclosure, then save the provider URL and send it to the training Telegram channel. The run record proves who approved the content and which job produced the asset.

- Brief: Audience, facts, CTA and duration.
- Approval: Script, identity and destination.
- Render: One video ID; bounded polling.
- Release: Checks, channel delivery and provenance.

## Lab 3 — Generate a Talking-Head Welcome Video

Learning outcome: LO3: Build and verify a two-phase n8n avatar-video workflow that binds human approval to an exact script hash before one asynchronous render.

Goal: You build a two-phase avatar-video workflow. A draft request turns approved facts into structured copy and returns the script plus its SHA-256 hash without calling HeyGen. A separate render request supplies the exact approved script and hash; n8n recomputes the hash, binds it to an authorised campaign profile, creates one video job and polls only that known job. A status-only starter lets an operator resume after the polling deadline without creating again.

Suggested duration: 55 minutes.

## Prerequisites

- Labs 1 and 2 are complete and the approved HarbourStay service facts are available as synthetic source material.
- A learner-owned OpenAI model credential and HeyGen API credential are stored in n8n; no secret appears in exported JSON.
- An authorised HeyGen avatar ID and voice ID are available, with quota for one short 1280x720 training render.
- Python 3 is available to serve labs/apps/video-studio; on Windows, `py -m http.server` is an equivalent fallback.

## What you'll build

A draft-and-render n8n webhook, a browser Video Studio that preserves the reviewed script snapshot, one authorised 16:9 captioned avatar video, a status-only resume workflow, and redacted evidence linking brief, script hash, profile version, video ID, attempts and terminal state. (Tools: n8n Webhook, AI Agent, OpenAI Chat Model, Code, HTTP Request, HeyGen API, Wait, IF, Respond to Webhook, browser Video Studio.)

## Step-by-step

1. Copy the evidence template to `evidence/lab03-avatar-video.md`. Record only synthetic content, hash and provider-ID suffixes; never paste complete credentials or private provider responses.

# Windows PowerShell

```
Copy-Item labs\evidence-template.md evidence\lab03-avatar-video.md
```

# macOS / Linux / WSL2

```
cp labs/evidence-template.md evidence/lab03-avatar-video.md
```

2. Import the supplied workflow and rename it C500 Lab 03 - Avatar Video. Confirm Avatar Video Webhook uses POST path `c500-avatar-video` and Respond to Webhook. Copy its Test URL.

```
labs/workflows/lab03-avatar-video.json
```

```
POST /webhook-test/c500-avatar-video
```

3. Open Validate Request. Confirm action must be draft or render. Draft requires `brief_id`, `audience`, `objective`, `approved_facts`, `call_to_action` and `max_words` 30-90. Render requires `brief_id`, `approved_script`, `approved_script_hash`, `approved_disclosure` and `approved_for_render` true.

```
action: draft | render
```

```
draft max_words: 30..90
```

```
render approved_for_render: true
```

4. Inspect Is Draft Request. Its true path alone reaches Script Agent; its false path goes directly to Validate Approved Draft. There must be no path from a render request back through the model.

```
draft -> Script Agent
```

```
render -> Validate Approved Draft
```

```
render -> Script Agent: forbidden
```

5. Attach a learner-owned OpenAI Chat Model to Script Agent. Keep its contract: use only approved\_facts and return title, spoken\_script, caption, claim\_list and disclosure; do not invent prices, availability, testimonials or guarantees.

```
Output: title, spoken_script, caption, claim_list, disclosure
```

6. Inspect Build Draft Snapshot and Hash Draft Script. Confirm the Code node extracts JSON even when the model wraps it in backtick or tilde fences, checks word count and claim subset, and requires disclosure. The native n8n Crypto node then writes a SHA-256 script\_hash over the exact spoken\_script without Code-node module imports.

```
script_contract_ok = true
```

```
Hash Draft Script: SHA256(exact spoken_script) -> script_hash
```

7. Open Return Draft. Confirm it returns status draft\_ready, brief\_id, exact spoken\_script, script\_hash, caption, disclosure, claim\_list and profile\_id. It must not run Campaign Profile or Create Avatar Video.

```
Expected draft status: draft_ready
```

```
Expected HeyGen create calls: 0
```

8. Open Video Studio through a local server, paste the Test URL and click Generate Draft. If python is unavailable on Windows, use the py fallback. Keep the n8n workflow listening.

```
python -m http.server 8001 --directory labs/apps/video-studio
```

```
# Windows fallback
```

```
py -m http.server 8001 --directory labs/apps/video-studio
```

```
Open http://localhost:8001
```

**9.** Review the displayed script, disclosure and hash. Use only the Approve Exact Draft & Render button; it must send the stored spoken\_script, disclosure and script\_hash without asking the model for another script.

```
action = render
```

```
approved_script = exact draft text
```

```
approved_script_hash = displayed SHA-256
```

```
approved_disclosure = exact draft disclosure
```

```
approved_for_render = true
```

**10.** Before a paid render, change one character in approved\_script with browser developer tools or an API client while keeping the old hash. Confirm Hash Approved Script and Approval Snapshot Valid route to approval\_mismatch and Create Avatar Video does not run.

```
Expected: approval_mismatch
```

```
Expected HeyGen create calls: 0
```

**11.** Open Campaign Profile. Replace <HEYGEN\_AVATAR\_ID> and <HEYGEN\_VOICE\_ID> with authorised assets. Keep profile\_version c500-demo-1, width 1280, height 720, captions true and destination private-preview.

```
avatar_id = <HEYGEN_AVATAR_ID>
```

```
voice_id = <HEYGEN_VOICE_ID>
```

```
profile_version = c500-demo-1
```

**12.** Select an n8n HTTP Header Auth credential for HeyGen with header X-API-Key in Create Avatar Video and Check Video Status. Confirm Create uses the exact approved\_script followed by the approved synthetic-media disclosure, authorised IDs, 1280x720 dimensions and captions.

```
POST https://api.heygen.com/v2/video/generate
```

```
Header: X-API-Key from n8n credential
```

```
voice.input_text = approved_script + approved_disclosure
```

**13.** Inspect both HTTP nodes. Continue On Fail or the error output must route provider errors to Normalise Provider Error, which returns review\_required with error\_class and any known video\_id rather than stopping before evidence is written.

```
create/status error -> Normalise Provider Error -> Return Provider Error
```

**14.** Click Approve Exact Draft & Render once. Confirm Store Job records brief\_id, approved\_script\_hash, approved\_disclosure, profile\_version, returned video\_id, poll\_attempt 0 and created\_at before Wait. Never retry Create Avatar Video.

```
Expected create calls: 1
```

Required stored fields: video\_id and approved\_disclosure

poll\_attempt = 0

- 15.** Trace Wait for Render, Check Video Status and Route Status. Processing increments poll\_attempt and returns to Wait; completed returns the URLs; failed and the 20-attempt deadline return review\_required while preserving the known video\_id.

Wait: 15 seconds

Maximum status attempts: 20

processing -> Wait; never -> Create

- 16.** If the deadline is reached, import C500 Lab 03 - Status Resume, set Known Job.video\_id to the retained value, select the HeyGen credential and run it. Confirm this workflow contains no create endpoint and performs status checks only.

labs/workflows/lab03-status-resume.json

GET /v1/video\_status.get?video\_id=<known video\_id>

Create endpoint count: 0

- 17.** For a completed job, compare the rendered speech and captions with the exact approved\_script; confirm authorised avatar, voice, 16:9 framing, disclosure and no unsupported claim. Record discrepancies as review\_required.

Release evidence: script hash, identity, voice, captions, framing, disclosure

- 18.** Activate the main workflow and replace the Video Studio URL with its Production URL. Save a final node-by-node diff checklist in evidence: imported node names, changed credential selectors, avatar ID, voice ID, webhook URL, and no other structural changes.

POST /webhook/c500-avatar-video

Diff: credentials + authorised IDs + URL only

## Test it

Draft mode returns a valid structured script and SHA-256 hash with zero HeyGen calls. A changed script with the old hash fails closed. The exact approved snapshot creates one job, stores its video\_id before bounded status polling, and ends completed or review\_required without duplicate creation. A timed-out job can resume through the status-only workflow, and completed evidence binds the exact script hash, profile version, video ID and final asset without exposing credentials.

## Checkpoint

Keep the credential-free main and status-only workflows, authorised campaign profile, approved script snapshot and completed Lab 3 evidence. Rejoin here: import labs/workflows/lab03-avatar-video.json, select OpenAI and HeyGen credentials, set

Campaign Profile, generate a fresh draft, approve that exact hash, then resume from Store Job using labs/workflows/lab03-status-resume.json only if a known video\_id has timed out.

### Troubleshooting

- The approved render produces a different script — Stop: render requests must bypass Script Agent. Send the exact draft as approved\_script and compare its recomputed SHA-256 with approved\_script\_hash before Campaign Profile.
- Create Avatar Video returns an error — Inspect Normalise Provider Error, compare the current official schema, confirm authorised IDs and preserve review\_required evidence without retrying create.
- Status remains processing beyond the lab — Retain the original video\_id and use lab03-status-resume.json; never submit the brief to the create endpoint again.
- Video content differs from the snapshot — Mark review\_required, preserve both artefacts and create a separately approved new snapshot rather than editing evidence.

### Challenge

Add a second approved 1080x1920 campaign profile. Bind its version into the approval snapshot and prove an unknown profile or mismatched version fails before the create call.

### Reflection

Which exact values must be frozen at approval so a later render is demonstrably the media a reviewer authorised?

**Note:** Full commands are in labs/lab-03-\*.md. Use only synthetic HarbourStay data, voices, avatars, bots and channels that you are authorised to use. Keep secrets in n8n credentials, disclose synthetic media, and do not publish a generated asset outside the private training channel.

## Lab 4 — Automate a Governed Video Campaign Pipeline

Learning outcome: LO4: Automate an approved script snapshot through persistent idempotency, avatar rendering, release checks and private-channel distribution with recoverable operating evidence.

Goal: You operationalise the Lab 3 contract with two n8n Data Tables. The queue stores an exact approved script hash, campaign profile version and destination; the run ledger persists an idempotency key before any paid render. Provider errors, failed release checks and timeouts update the same ledger row to review\_required with a known video ID when available. A replay stops at the persistent existence check.

Suggested duration: 50 minutes.

## Prerequisites

- Lab 3 is complete with an approved script snapshot, authorised profile and a verified create-once/status-only contract.
- A private Telegram training channel exists and the learner-owned bot is an administrator; its destination ID is reviewed but not committed to Git.
- The n8n workspace provides Data Tables and the HeyGen and Telegram credentials used in the prior labs.
- Quota is available for no more than one additional short render unless the trainer explicitly authorises more.

## What you'll build

A c500\_campaign\_queue approval table, c500\_campaign\_runs ledger, scheduled-but-disabled n8n pipeline, one privately distributed training video, a duplicate-skip trace, status-only recovery path and an operating runbook. (Tools: n8n Schedule Trigger, Manual Trigger, Data Table, Code, HeyGen API, Wait, IF, Telegram Send Video, persistent ledger, release runbook.)

## Step-by-step

1. Copy the evidence template to evidence/lab04-campaign-pipeline.md and create evidence/video-agent-runbook.md. Use synthetic identifiers and name a learner operator plus trainer release reviewer.

```
# Windows PowerShell
```

```
Copy-Item labs\evidence-template.md evidence\lab04-campaign-pipeline.md
```

```
New-Item -ItemType File -Path evidence\video-agent-runbook.md -Force
```

```
# macOS / Linux / WSL2
```

```
cp labs/evidence-template.md evidence/lab04-campaign-pipeline.md
```

```
: > evidence/video-agent-runbook.md
```

2. Create Data Table c500\_campaign\_queue with the exact columns below. approved\_script, approved\_script\_hash, approved\_profile\_version and approved\_destination\_id form the trusted release snapshot.

```
campaign_id:string
```

```
approved_script:string
```

```
approved_script_hash:string
```

```
approved_profile_version:string
```

```
approved_destination_id:string
```

approval\_status:string

requested\_by:string

3. Create Data Table c500\_campaign\_runs with the exact persistent ledger columns. request\_key is the replay guard; status and error\_class preserve terminal or review state.

campaign\_id:string

request\_key:string

status:string

video\_id:string

video\_url:string

destination\_id:string

message\_id:string

attempts:number

script\_hash:string

profile\_version:string

started\_at:dateTime

completed\_at:dateTime

error\_class:string

4. Add one queue row from the exact approved Lab 3 snapshot. Set campaign\_id c500-weekly-tip-001, copy the approved script and SHA-256 exactly, use profile version c500-demo-1, your private destination ID and approval\_status pending.

campaign\_id: c500-weekly-tip-001

approved\_script: <exact Lab 3 script>

approved\_script\_hash: <exact SHA-256>

approved\_profile\_version: c500-demo-1

approved\_destination\_id: <private channel ID>

approval\_status: pending

5. Import C500 Lab 04 - Video Campaign Pipeline. Keep Schedule Trigger disabled and use Manual Trigger. Confirm Get Approved Campaign is an actual Data Table Row/Get operation filtered to campaign\_id c500-weekly-tip-001.

labs/workflows/lab04-video-campaign-pipeline.json

Table: c500\_campaign\_queue

Operation: Row / Get

6. Inspect Approval Gate. Only approval\_status exactly approved reaches Pipeline Configuration and the persistent run query. Pending content routes to Record Unapproved Skip and stops before HeyGen or Telegram.

pending -> skipped\_unapproved

approved -> validate snapshot -> query ledger

7. Inspect Find Existing Run. It is a Data Table Row/Get operation on c500\_campaign\_runs filtered by request\_key c500-video:<campaign\_id>, limited to one row and configured to output an empty item when no row exists. Classify Existing Run routes an existing row to Record Duplicate Skip and a no-row result to Insert Run Ledger.

request\_key = c500-video:<campaign\_id>

existing row -> skipped\_duplicate

no row -> Insert Run Ledger

8. Run the pending row. Confirm Insert Run Ledger, Create Avatar Video and Publish Private Preview do not execute. Record skipped\_unapproved and zero external calls.

Expected external provider calls: 0

Expected ledger inserts: 0

9. Review the exact script, hash, profile version and private destination, then change only approval\_status to approved. Open Pipeline Configuration and set authorised avatar\_id, voice\_id, profile\_version c500-demo-1, poll\_seconds 15 and maximum\_poll\_attempts 20.

approval\_status: approved

profile\_version: c500-demo-1

maximum\_poll\_attempts: 20

10. Inspect Validate Approved Snapshot and Hash Queue Script. The Code node checks required snapshot fields and profile version; the native n8n Crypto node writes SHA-256 into script\_hash. Approved Snapshot Valid requires exact equality with approved\_script\_hash and a non-empty approved\_destination\_id. A mismatch stops before ledger insert and provider calls.

Hash Queue Script: SHA256(approved\_script) -> script\_hash

script\_hash = approved\_script\_hash

approved\_profile\_version = configured profile\_version

**11.** Select HeyGen credentials in Create Avatar Video and Check Video Status. Confirm Insert Run Ledger writes request\_key and status rendering before Create, and Store Provider Video ID updates that same row immediately after a successful create response.

`insert status: rendering`

`create calls: at most 1`

`update video_id: before Wait`

**12.** Inspect error wiring. Create and status HTTP error outputs, failed state, timeout and failed Release Checks all reach Mark Review Required, which updates the same request\_key with status review\_required, error\_class and the known video\_id when available.

`error -> review_required`

`known video_id retained`

`never route error to Create`

**13.** Inspect Wait, Check Video Status and Route Status. Processing increments attempts and loops to Wait; completed goes to Release Checks; failed or attempt 20 goes to Mark Review Required. No loop returns to Create Avatar Video.

`processing -> Wait`

`completed -> Release Checks`

`failed|timeout -> Mark Review Required`

**14.** Inspect Hash Release Script and Release Checks. The native Crypto node recomputes the script hash; Release Checks requires completed status, non-empty video\_url, exact hash and profile matches, then passes through only the trusted approved\_destination\_id from the queue snapshot. No provider or publish response may replace that destination.

`Required: completed + video_url + exact hash + exact profile`

`Publish destination source: approved queue snapshot only`

**15.** Open Publish Private Preview. Select the Telegram credential, Send Video by URL, Chat ID from approved\_destination\_id and a caption that includes synthetic-media disclosure. Its error output must also reach Mark Review Required.

`Video: {{$json.video_url}}`

`Chat ID: {{$json.approved_destination_id}}`

`Caption suffix: Generated with an AI avatar - training preview.`

**16. Inspect Mark Published.** It updates the existing `c500_campaign_runs` row filtered by `request_key` with status published, video ID and URL, Telegram message ID, attempts, exact hash, profile version and `completed_at`.

Operation: `Data Table Row / Update`

Filter: `request_key`

`status: published`

**17. Run the approved row once.** Observe one ledger insert, one create call, the known video ID through bounded polling, one private disclosed Telegram post and one published update.

Expected ledger rows for `request_key`: 1

Expected create calls: 1

Expected Telegram posts: 1

**18. Replay with Manual Trigger.** Confirm Find Existing Run returns the persisted row and Record Duplicate Skip executes; Insert Run Ledger, HeyGen and Telegram must not execute. Save the execution trace.

Expected: `skipped_duplicate`

Additional create calls: 0

Additional Telegram posts: 0

**19. Complete the runbook with owners, source and snapshot rules, schedule, maximum one video per run, poll deadline, monthly usage limit, monitoring, status-only recovery, credential rotation, retention, private-to-public review and rollback. Leave Schedule Trigger disabled. Save a node-by-node diff checklist naming only credential selectors, authorised IDs and Data Table bindings changed from the starter.**

Recovery: `labs/workflows/lab03-status-resume.json`

Timezone: `Asia/Singapore`

Example schedule: `Monday 09:00`

Final schedule state: `disabled`

Diff: `tables + credentials + authorised IDs only`

## Test it

Pending content causes zero ledger inserts and zero external calls. The exact approved snapshot inserts one persistent request key before one create call, stores the returned video ID, reaches published or `review_required` through bounded polling, and writes every error path to the ledger. A published run sends one disclosed private video, while replay resolves through the persistent existing-row query with zero additional render or

Telegram calls. A known timed-out job can be checked through the status-only workflow without calling create.

## Checkpoint

Leave Schedule Trigger disabled and retain the synthetic queue row, persistent run-ledger row, private preview, duplicate-skip trace and runbook. Rejoin here: import labs/workflows/lab04-video-campaign-pipeline.json, bind both Data Tables and credentials, set Pipeline Configuration, confirm approval snapshot hash, then run Manual Trigger; for any retained video\_id use labs/workflows/lab03-status-resume.json and never rerun Create.

## Troubleshooting

- A pending row reaches HeyGen — Stop the run and confirm Approval Gate is before Validate Approved Snapshot, both existence checks, ledger insert and every external provider node.
- A replay creates another video — Disable the workflow, confirm request\_key is identical, bind both Data Table existence operations to c500\_campaign\_runs and restore the rowNotExists/rowExists branches before Create.
- Ledger remains rendering after a provider error — Connect the HTTP error output to Mark Review Required and update the same request\_key with error\_class plus any known video\_id.
- Telegram cannot send the video URL — Keep the run ledger, mark review\_required, verify bot access and approved\_destination\_id, then retry only the publish stage after human review; never create another video.

## Challenge

Add an approval-expiry timestamp to the trusted queue snapshot and prove an expired row fails before ledger insert, HeyGen or Telegram.

## Reflection

What production database constraint would make the persistent request\_key guarantee atomic when two scheduler executions start at the same instant?

**Note:** Full commands are in labs/lab-04-\*.md. Use only synthetic HarbourStay data, voices, avatars, bots and channels that you are authorised to use. Keep secrets in n8n credentials, disclose synthetic media, and do not publish a generated asset outside the private training channel.

## Topic recap

- You can now: LO3: Explain avatar-video architecture and build a governed brief-to-script-to-asynchronous-render workflow.

- You can now: LO4: Operate an approved, idempotent video pipeline with release checks, private distribution, monitoring and rollback.

## Wrap-Up and Authoritative References

You now have one reusable conversation core, two channel patterns and one governed asynchronous video pipeline. Recheck the official documentation before a production deployment because node and provider interfaces evolve.

### Official n8n references

- Webhook node: <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.webhook/>
- AI Agent node: <https://docs.n8n.io/integrations/builtin/cluster-nodes/root-nodes/n8n-nodes-langchain.agent/>
- HTTP Request node: <https://docs.n8n.io/integrations/builtin/core-nodes/n8n-nodes-base.httprequest/>
- Telegram node: <https://docs.n8n.io/integrations/builtin/app-nodes/n8n-nodes-base.telegram/>

### Official provider references

- OpenAI speech to text: <https://platform.openai.com/docs/guides/speech-to-text>
- ElevenLabs text to speech: <https://elevenlabs.io/docs/api-reference/text-to-speech/convert>
- HeyGen create avatar video: <https://docs.heygen.com/reference/create-an-avatar-video-v2>
- HeyGen video status: <https://docs.heygen.com/reference/video-status>
- Telegram Bot API: <https://core.telegram.org/bots/api>

### Production readiness questions

- Who owns each agent outcome, provider account, approved identity, destination and incident response?
- Which inputs, actions and channels are explicitly forbidden, and how is human handoff verified?
- What evidence proves factual accuracy, accessibility, consent, latency, cost and delivery?
- How are credentials rotated, media retained, duplicates prevented and published content corrected or removed?

## Next Steps

- Replace the synthetic service facts with one approved, versioned knowledge source and retain its provenance.
- Extract the conversation core and video-render loop into reusable n8n sub-workflows with compact input/output contracts.
- Add dashboards for stage latency, failure classes, human handoffs, duplicate prevention and cost per accepted media asset.
- Pilot with internal users and private channels before enabling telephone numbers or public distribution.

## Glossary

- **AI Agent** — A component that interprets an objective and chooses permitted actions within a defined role and workflow boundary.
- **Avatar** — The authorised visual identity rendered as the presenter in a synthetic video.
- **Binary data** — Non-text payload such as audio or video carried in a named n8n binary property.
- **Channel adapter** — Workflow logic that maps a browser, chat or phone event to and from the shared conversation contract.
- **Idempotency key** — A stable identifier used to prevent the same event from creating the same side effect more than once.
- **Speech to text** — Transcribing an audio signal into written words; in this course the provider pattern uses Whisper.
- **Synthetic media** — Audio or video generated or materially altered by an AI system.
- **Text to speech** — Synthesising spoken audio from approved response text; in this course the provider pattern uses ElevenLabs.
- **Webhook** — An HTTP endpoint that starts or continues a workflow when another system sends an event.